

ui.linear_progress 全面详解

`ui.linear_progress` 是 NiceGUI 框架中封装 Quasar 组件的线性进度条组件，用于直观展示任务进度、加载状态或数值占比（范围 0.0-1.0）。其核心优势是轻量化、配置简洁，支持样式定制、值绑定、进度变化监听等功能，无缝适配 Web 界面的进度展示场景（如文件上传、任务执行、数据加载等）。以下从核心特性、使用场景、详细配置、高级技巧等维度展开全面解析。

一、核心概述

1. 本质与定位

- 基于 Quasar 的 `QLinearProgress` 组件封装，专注于线性进度可视化，无需复杂配置即可实现进度展示。
- 支持进度值动态更新、样式自定义（颜色、高度）、进度文本显示，适配从简单加载提示到复杂进度监控的各类场景。
- 继承 NiceGUI 组件通用能力，支持值绑定、事件监听、可见性控制，轻松融入 Web 交互逻辑。

2. 核心参数（初始化时必须填 / 常用）

参数名	类型	说明
<code>value</code>	float	初始进度值（必填，范围 0.0-1.0，0 表示无进度，1 表示完成）。
<code>size</code>	str	进度条高度（默认：显示进度文本时为 "20px"，隐藏时为 "4px"），支持 CSS 长度单位（如 "8px"、"1rem"）。
<code>show_value</code>	bool	是否在进度条中心显示百分比文本（默认 <code>True</code> ，如 50%）。
<code>color</code>	str / None	进度条颜色（默认 "primary"），支持 Quasar 主题色（如 "secondary"、"accent"）、Tailwind 颜色（如 "bg-blue-500"）或 CSS 颜色（如 "#b687ac"），设为 <code>None</code> 时使用默认样式。
<code>style</code>	Style[Self]	进度条容器样式，支持 Tailwind CSS（如 <code>rounded-full</code> 、 <code>shadow-sm</code> ）。
<code>classes</code>	Classes[Self]	进度条容器的 HTML 类名，用于样式复用。
<code>visible</code>	bool	初始可见性（默认 <code>True</code> ），支持后续动态修改或绑定。

二、基础使用场景与示例

1. 基础静态进度条

直接设置 `value` 参数，快速创建静态进度条，适用于固定进度展示（如任务完成度概览）。

示例：50% 静态进度条

```
from nicegui import ui

# 基础配置：50% 进度、显示文本、默认颜色和高度
```

```

ui.linear_progress(value=0.5)

# 自定义配置: 75% 进度、隐藏文本、红色、高度 8px、圆角
ui.linear_progress(
    value=0.75,
    show_value=False,
    color='red',
    size='8px',
    style='rounded-full' # 圆角样式
)

ui.run()

```

2. 动态更新进度（定时器模拟加载）

通过 `set_value` 方法修改进度值，结合 `ui.timer` 模拟实时进度更新（如文件上传、数据加载）。

示例：模拟加载进度 (0-100%)

```

from nicegui import ui

# 初始化进度条（初始值 0，显示文本）
progress = ui.linear_progress(value=0.0, size='10px', color='#28738a')
status_label = ui.label('Loading... 0%')

# 模拟进度更新：每 200ms 增加 2%，直至 100%
current_value = 0.0
def update_progress():
    global current_value
    if current_value < 1.0:
        current_value += 0.02
        progress.set_value(min(current_value, 1.0)) # 更新进度值
        status_label.set_text(f'Loading... {int(current_value*100)}%')
    else:
        status_label.set_text('Loaded!')
        timer.deactivate() # 进度完成，停止定时器

# 启动定时器
timer = ui.timer(0.2, update_progress)

ui.run()

```

3. 与滑块绑定（实时控制进度）

通过 `bind_value_from` 方法将进度条与滑块组件绑定，实现进度的手动调节（如视频播放进度、音量控制）。

示例：滑块控制进度条

```

from nicegui import ui

# 创建滑块（范围 0-1，步长 0.01，初始值 0.3）
slider = ui.slider(min=0, max=1, step=0.01, value=0.3)

# 进度条绑定滑块值（滑块变化时，进度条自动更新）
ui.linear_progress().bind_value_from(slider, 'value')

# 双向绑定：进度条变化也同步到滑块（可选）
# progress = ui.linear_progress(value=0.3)
# progress.bind_value(slider, 'value') # 双向绑定

ui.run()

```

4. 进度变化事件监听

通过 `on_value_change` 方法绑定回调，监听进度值变化（如进度完成时触发提示、执行后续操作）。

示例：进度完成时弹出提示

```

from nicegui import ui

progress = ui.linear_progress(value=0.0, color='green')

# 监听进度值变化
def on_progress_change(e):
    current_value = e.value
    if current_value >= 1.0:
        ui.notify('Task completed successfully!', type='success')

progress.on_value_change(on_progress_change)

# 模拟进度更新
ui.timer(0.1, lambda: progress.set_value(min(progress.value + 0.05, 1.0)))

ui.run()

```

5. 自定义样式（颜色、圆角、阴影）

结合 `color`、`style`、`classes` 参数，定制进度条外观，适配 Web 界面设计风格。

示例：高颜值定制进度条

```

from nicegui import ui

# 1. Tailwind 颜色 + 圆角 + 阴影
ui.linear_progress(
    value=0.6,
    color='bg-purple-500', # Tailwind 颜色
    style='rounded-full shadow-md',
    size='12px'
)

```

```
# 2. CSS 渐变颜色（通过 style 自定义）
ui.linear_progress(
    value=0.8,
    show_value=False,
    size='15px',
    # 渐变背景：从蓝色到青色
    style='''
        background-color: #e5e7eb;
        border-radius: 8px;
        overflow: hidden;
    ''',
    # 进度条部分的渐变样式
    classes='''
        [role="progressbar"] {
            background: linear-gradient(90deg, #3b82f6, #06b6d4);
        }
    ''',
)

ui.run()
```

6. 可见性绑定（动态显示 / 隐藏）

通过 `bind_visibility` 方法将进度条可见性与其他组件状态绑定（如开关、复选框），适配复杂界面交互。

示例：开关控制进度条显示

```
from nicegui import ui

# 创建开关组件
show_progress = ui.switch('Show Progress', value=True)

# 创建进度条并绑定可见性
progress = ui.linear_progress(value=0.4, color='orange')
progress.bind_visibility(show_progress, 'value')

ui.run()
```

7. 多进度条并列（任务组进度）

多个进度条并列展示，适用于多任务进度监控（如批量文件上传、多步骤流程）。

示例：多任务进度监控

```
from nicegui import ui
import random

# 模拟 3 个任务的进度
tasks = [
    {'name': 'Task 1', 'value': 0.3},
    {'name': 'Task 2', 'value': 0.6},
```

```
        {'name': 'Task 3', 'value': 0.9}
    ]

    for task in tasks:
        # 任务名称 + 进度条 横向排列
        with ui.row(classes='items-center w-full mb-2'):
            ui.label(task['name'], classes='w-20')
            progress = ui.linear_progress(value=task['value'], size='8px')
            # 随机颜色
            colors = ['#b687ac', '#28738a', '#a78f8f', '#f1c40f']
            progress.color = random.choice(colors)

    ui.run()
```

三、核心属性与方法详解

1. 常用属性

属性名	类型	说明
value	BindableProperty	可读可写，进度值（0.0-1.0），修改后自动刷新进度条（如 <code>progress.value = 0.7</code> ）。
size	str	可读可写，进度条高度，支持动态修改（如 <code>progress.size = '10px'</code> ）。
show_value	bool	可读可写，是否显示进度文本，修改后需调用 <code>update()</code> 生效（如 <code>progress.show_value = False; progress.update()</code> ）。
color	str / None	可读可写，进度条颜色，支持动态修改（如 <code>progress.color = 'green'</code> ）。
visible	BindableProperty	可读可写，控制进度条是否可见（支持绑定，如 <code>bind_visibility</code> ）。
html_id	str	进度条在 HTML DOM 中的唯一 ID（v2.16.0+ 支持）。

2. 核心方法

方法名	作用	参数说明
set_value(value)	手动设置进度值（核心方法），值需在 0.0-1.0 范围内。	value：进度值（float，如 0.5 表示 50%）。
on_value_change(callback)	绑定进度值变化事件回调。	callback：接收 <code>ValueChangeEventArguments</code> 对象， <code>e.value</code> 为当前进度值。
bind_value(target)	双向绑定进度值到目标对象的属性（如滑块、变量）。	target_object：目标对象； target_name：属性名（默认 <code>value</code> ）。

方法名	作用	参数说明
<code>bind_value_from(target)</code>	单向绑定进度值（从目标对象到进度条）。	参数同 <code>bind_value</code> ，仅单向同步。
<code>bind_value_to(target)</code>	单向绑定进度值（从进度条到目标对象）。	参数同 <code>bind_value</code> ，仅单向同步。
<code>bind_visibility(target)</code>	双向绑定可见性到目标对象的属性（如开关）。	<code>target_object</code> ：目标对象； <code>target_name</code> ：属性名（默认 <code>visible</code> ）。
<code>update()</code>	手动触发进度条刷新（修改 <code>show_value</code> 、 <code>size</code> 等属性后需调用）。	-
<code>set_visibility(visible)</code>	直接设置可见性（布尔值）。	<code>visible</code> ： <code>True</code> （显示） / <code>False</code> （隐藏）。
<code>delete()</code>	删除进度条组件，释放资源。	-

四、高级技巧与注意事项

1. 样式定制进阶

- 渐变进度条**：通过 `style` 和 `classes` 自定义进度条背景渐变，提升视觉效果（参考“自定义样式”示例）。
- 响应式高度**：使用相对单位（如 `rem`、`%`）设置 `size`，适配不同屏幕尺寸（如 `size='2vw'` 表示屏幕宽度的 2%）。
- 进度条背景**：默认背景为浅灰色，可通过 `style` 修改（如 `style='background-color: #f3f4f6'`）。

2. 性能优化

- 高频更新控制**：若需高频更新进度（如每秒 10 次以上），可适当降低更新频率，或使用 `throttle` 限制事件触发次数：

```
# 限制进度变化事件 0.1 秒内仅触发一次
progress.on_value_change(callback, throttle=0.1)
```

- 避免不必要刷新**：修改 `value` 时直接赋值（`progress.value = 0.6`）或调用 `set_value`，无需额外调用 `update()`；修改 `show_value`、`size` 等属性后才需调用 `update()`。

3. 兼容性与版本说明

- 颜色兼容性**：`color` 参数支持 Quasar 主题色、Tailwind 颜色、CSS 颜色，建议优先使用 CSS 颜色（如 `#xxx`、`rgb()`）确保跨环境兼容。
- 版本要求**：`html_id` 属性需 NiceGUI v2.16.0+ 支持，低版本需避免使用。
- 容器样式**：修改 `style` 时需注意 `overflow: hidden` 对圆角的影响（需同时设置进度条和容器的圆角）。

4. 常见问题排查

- **进度条不更新**: 检查 `value` 是否在 0.0-1.0 范围内, 修改属性后是否调用 `update()` (如 `show_value` 变化后)。
- **颜色不生效**: 确认颜色值格式正确 (如 Tailwind 颜色需带 `bg-` 前缀, CSS 颜色需符合语法), 避免拼写错误。
- **文本显示异常**: `show_value=True` 时, 进度条高度需足够 (默认 20px), 若 `size` 过小 (如 4px), 文本可能被截断, 建议隐藏文本或增大高度。

五、总结

`ui.linear_progress` 是 NiceGUI 中轻量化、高易用性的进度展示组件, 核心优势在于:

1. 配置简洁: 通过少量参数即可实现进度条的样式与功能定制, 无需复杂代码。
2. 动态交互: 支持进度值更新、事件监听、组件绑定, 适配实时进度监控场景。
3. 样式灵活: 兼容 Quasar、Tailwind、CSS 颜色与样式, 可无缝融入各类 Web 界面。
4. 轻量化: 组件体积小, 渲染性能优, 适合批量使用 (如多任务进度监控)。

适用于文件上传、任务执行、数据加载、视频播放进度、多步骤流程等场景, 是 Web 应用中进度可视化的首选组件。